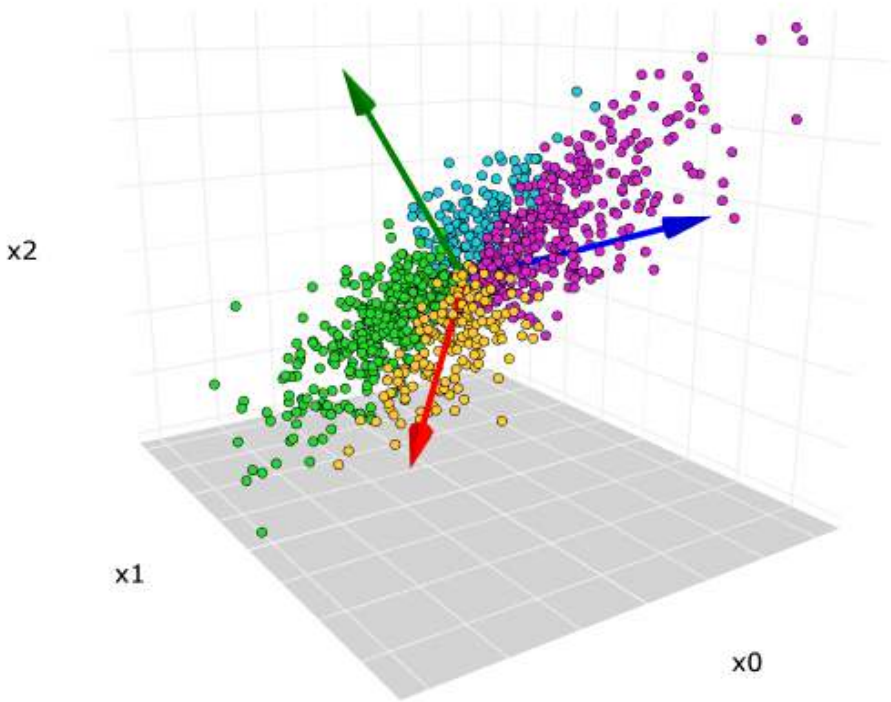
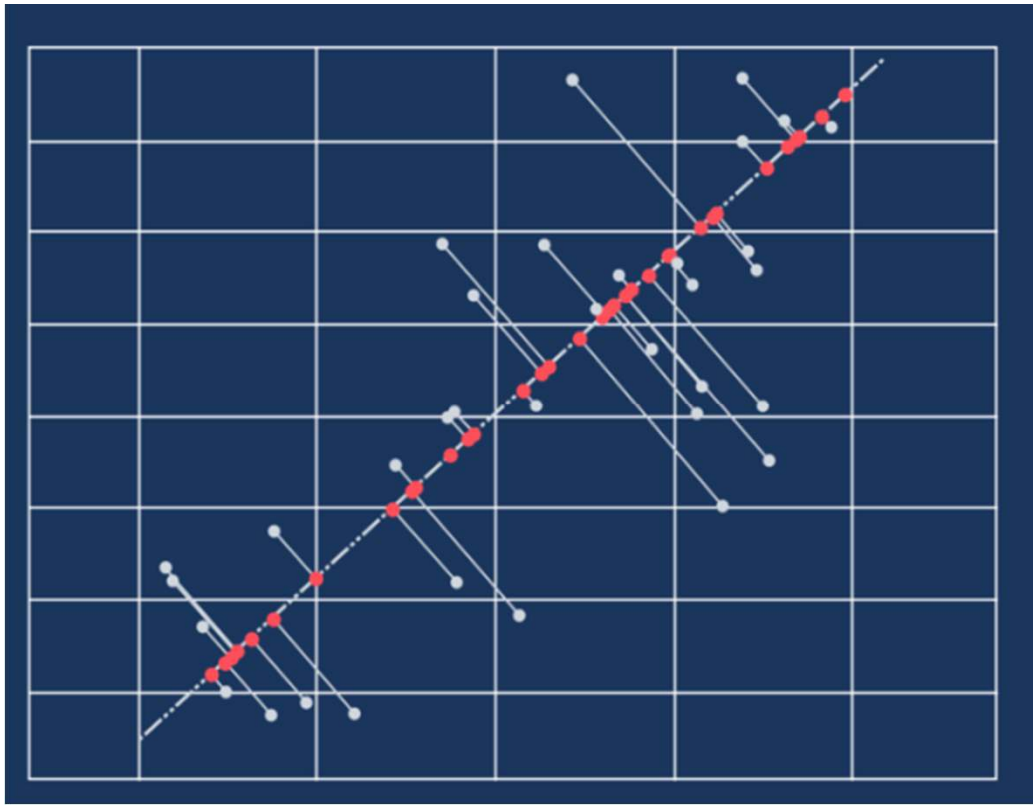


$$Cu = \lambda u$$

$$|C - \lambda I| = 0$$



ANALISIS KOMPONEN UTAMA (PRINCIPAL COMPONENT ANALYSIS)

MUHAMMAD DJAMALUDDIN

TENTANG SLIDE

Bahan :

https://bipk.uma.ac.id/2024/06/28/dimensi-tinggi-dalam-pembelajaran-mesin/

https://builtin.com/data-science/step-step-explanation-principal-component-analysis

https://www.youtube.com/watch?v=FD4DeN81ODY&t=16s

DATA BERDIMENSI TINGGI

- Data berdimensi tinggi (high-dimensional data) mengacu pada dataset yang memiliki sejumlah besar fitur atau variabel.

- Dalam konteks machine learning, data berdimensi tinggi seringkali menimbulkan tantangan tersendiri karena peningkatan kompleksitas dan kebutuhan komputasi yang lebih besar.

- **Karakteristik Data Berdimensi Tinggi**

✓ **Jumlah Fitur yang Besar:** Data berdimensi tinggi memiliki banyak fitur (dimensi) dibandingkan dengan jumlah observasi atau sampel.

✓ **Ruang Fitur yang Luas:** Dengan bertambahnya jumlah fitur, volume ruang fitur tumbuh secara eksponensial, menyebabkan data menjadi sangat tersebar.

✓ **Sparsity (Kerapatan Rendah):** Data berdimensi tinggi seringkali jarang, yang berarti sebagian besar entri dalam dataset adalah nol atau tidak ada.

CONTOH DATA BERDIMENSI TINGGI

- **Pemrosesan Gambar dan Video:**

Sebuah gambar terdiri dari banyak piksel dari ratusan sampai jutaan dan setiap piksel mewakili suatu fitur, sehingga gambar menjadi data berdimensi tinggi.

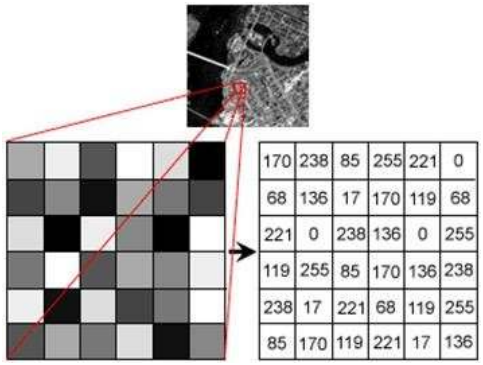
Video adalah deretan Gambar, 1 detik bisa 30-60 frame. Sehingga data video adalah data berdimensi tinggi

- **Data Kesehatan:**

Catatan medis pasien dapat berisi sejumlah informasi seperti tekanan darah, detak jantung, tinggi badan, berat badan, dan berbagai parameter medis lainnya.

- **Data Keuangan:**

Kumpulan data keuangan sebuah perusahaan dapat mencakup fitur seperti kapitalisasi pasar, volume perdagangan, rasio PE, tingkat dividen, dll.



features

observations

	Blood pressure	Heart rate	height	weight	...	...
Person 1						
Person 2						
Person 3						

Consolidated Balance Report

Xtreme Mountain Bikes, Inc.

For the last 4 Quarters

Assets

	Qtr1	Qtr2	Qtr3	Qtr4
Cash	$932,694	$1,116,535	$1,1328,181	$505,228
Account Receivable	$567,113	$353,221	$462,197	$867,008
Inventories	$313,714	$699,555	$423,018	$893,008
Other Current assets	$5,000	$4,0000	$5,000	$5,000
Total Current Assets	**$1,818,521**	**$2,173,311**	**$2,218,396**	**$2,270,244**
Land	$222,909	$223,453	$223,541	$223,755
Buildings	$813,799	$813,835	$813,769	$813,797
Equipments etc	$321,491	$341,549	$451,591	$601,600
Less Accumulated Depreciation	$0	$0	$0	$0
Total Net Fixed Assets	**$1,358,199**	**$1,378,837**	**$1,488,901**	**$1,639,152**
Other assets	$123,377	$143,545	$145,392	$183,468

TANTANGAN DATA BERDIMENSI TINGGI

1. **Kutukan Dimensi (*Curse of Dimensionality*):** Dengan peningkatan dimensi, volume ruang fitur meningkat begitu cepat sehingga data menjadi sangat jarang. Hal ini membuat algoritma machine learning lebih sulit karena model memerlukan lebih banyak data untuk mencapai performa yang diinginkan.

2. **Overfitting:** Model yang dilatih pada data berdimensi tinggi cenderung overfit jika jumlah data sedikit, yaitu model belajar dengan sangat baik dari data pelatihan tetapi performanya buruk pada data testing. Banyaknya fitur meningkatkan kemungkinan model menangkap derau (noise) sebagai sinyal.

3. **Kebutuhan Komputasi:** Pengolahan data berdimensi tinggi membutuhkan sumber daya komputasi yang besar baik dalam hal memori maupun waktu pemrosesan.

4. **Kesulitan Visualisasi:** Visualisasi data berdimensi tinggi sangat sulit karena keterbatasan manusia dalam memvisualisasikan lebih dari tiga dimensi.

PENANGANAN DATA BERDIMENSI TINGGI

Dalam machine learning, ada beberapa cara menangani data berdimensi tinggi , seperti:

1. **Seleksi Fitur (Feature Selection):** Teknik untuk memilih subset fitur yang paling relevan dengan target prediksi, mengurangi dimensi data tanpa kehilangan informasi fitur yang penting untuk permasalahan yang ada.

2. **Reduksi Dimensi (Dimensional Reduction):** Teknik untuk mengubah data berdimensi tinggi menjadi representasi dimensi yang lebih rendah tanpa kehilangan hampir semua informasi yang penting

3. **Regularisasi :** Mengatur kompleksitas model sehingga tidak rentan pada problem overfitting

4. **Augmentasi Data** : Menambahkan data sintesis untuk menambah jumlah sampel sehingga rasio dimensi data vs jumlah sampel berkurang dan memperkaya variasi data.

Kombinasi dari berbagai cara tersebut terkadang dilakukan sesuai dengan spesifik masalah yang dihadapi

SELEKSI FITUR

- Seleksi Fitur adalah proses menyeleksi subset terbaik dari himpunan semua fitur yang tersedia.
- Subset fitur terbaik menunjukkan jumlah fitur terkecil yang berkontribusi pada performa model
- Jika A adalah himpunan fitur pada data, dan $|A| = d$, maka terdapat 2^d subset fitur yang mungkin.
- Ada 2 pendekatan :

1. Seleksi maju (***Forward selection***) : mulai dari 0 fitur dan ditambah satu per-satu, dicek mana yang memberi paling besar error sampai penambahan fitur tidak mengurangi error
2. Seleksi mundur (***Backward selection***) : mulai dari d fitur dan dikurangi satu persatu, dicek mana yang memberikan pengurangan error terbesar

Pengetesan error dilakukan pada data validasi bukan pada data pelatihan

REDUKSI DIMENSI

- Dalam banyak algoritma pembelajaran, kompleksitas tergantung pada jumlah dimensi input, $\boldsymbol{d}$, serta pada ukuran sampel data, $\boldsymbol{N}$.
- Reduksi dimensi mengurangi memori dan tenaga komputasi yang diperlukan. Penurunan $\boldsymbol{d}$ juga mengurangi kompleksitas algoritma inferensi selama pengujian.
- Ketika bagian dari input dinyatakan tidak perlu, bisa menghemat biaya ekstraksi.
- Model yang lebih sederhana, lebih tangguh pada data berjumlah kecil. Model yang lebih sederhana memiliki varians yang lebih sedikit, artinya terjadi pengurangan variasi tergantung pada jenis sampel, termasuk noise, outlier, dan sebagainya.

REDUKSI DIMENSI

- Ketika data dapat dijelaskan dengan lebih sedikit fitur, maka muncul ide yang lebih baik tentang proses yang mendasari data dan proses ekstraksi-nya.
- Fitur yang lebih sedikit ini dapat ditafsirkan sebagai fitur tersembunyi atau faktor-faktor laten yang dalam dikombinasikan menghasilkan fitur yang diamati.
- Ketika data dapat direpresentasikan dalam beberapa dimensi tanpa kehilangan informasi, maka data dapat diplot dan dianalisis secara visual untuk melihat struktur dan mendeteksi outliers.

PRINCIPAL COMPONENT ANALYSIS

- **Analisis Komponen Utama (AKU)** atau ***Principal Component Analysis*** **(PCA)** adalah sebuah metode untuk mereduksi data dimensi tinggi ke dimensi lebih rendah dengan memperhatikan komponen terpenting dari data dimensi tinggi tersebut
- PCA ditemukan pertama kali oleh Karl Pearson di tahun 1901
- Kita ingin melakukan pemetaan dari data dengan input berdimensi d ke dimensi k, di mana $k < d$, tetapi dengan syarat informasi yang hilang minimum.
- PCA bertujuan untuk mengidentifikasi kerangka acuan yang paling berarti (juga disebut ***basis*** dalam aljabar linier) untuk mengekspresikan kembali dataset dengan harapan bahwa basis baru ini akan menyaring data derau (noise) dan mengungkapkan struktur tersembunyi.
- Jadi prinsip utama PCA adalah menemukan vektor basis yang dapat merepresentasikan data dengan baik
- Karena PCA bersifat unsupervised, tidak ada label data sebagai penuntun perbaikan error, maka kriteria yang dimaksimalkan adalah *variance*.

VEKTOR DASAR (BASIS VECTOR)

- Ruang Vektor (***Vector Space***) adalah struktur matematis berbentuk himpunan vektor yang dapat melakukan operasi penjumlahan dan perkalian skalar dan memenuhi beberapa sifat tertentu.
- Dalam ruang vektor, jika satu set vektor dapat digunakan untuk mengekspresikan setiap vektor dalam ruang sebagai kombinasi linier unik dari vektor tersebut, dan vektor tersebut secara linier independen (artinya tidak ada yang dapat dinyatakan sebagai kombinasi linier dari yang lain), maka kita menyebutnya sebagai ***vektor dasar (basis vector)*** untuk ruang vektor tersebut.
- Jumlah vektor dasar pada sebuah ruang vektor ditentukan dimensi dari ruang vektor. Jika $V \in \mathbb{R}^3$, maka terdapat 3 vektor dasar.
- Contoh dalam ruang kartesian $V = \{(x, y) | x, y \in \mathbb{R}\}$ kita memiliki 2 vektor dasar $B = \{\begin{bmatrix}1\\0\end{bmatrix}, \begin{bmatrix}0\\1\end{bmatrix}\}$, semua vektor dalam V bisa dianggap kombinasi linier yang unik dari B.
- Contoh $v_1 = \begin{bmatrix}0.7\\-1\end{bmatrix}$ bisa ditulis sebagai $v_1 = 0.7 \begin{bmatrix}1\\0\end{bmatrix} + (-1) \begin{bmatrix}0\\1\end{bmatrix}$

  $v_2 = \begin{bmatrix}10\\4\end{bmatrix}$ bisa ditulis sebagai $v_2 = 10 \begin{bmatrix}1\\0\end{bmatrix} + 4 \begin{bmatrix}0\\1\end{bmatrix}$

CARA KERJA PCA

- Prinsip dari PCA adalah menggunakan komponen utama untuk mengompres dimensi.
- Komponen utama adalah vector basis yang memiliki arah yang memaksimalkan varians data
- Proyeksi x pada sebuah vector basis u dinyatakan dengan

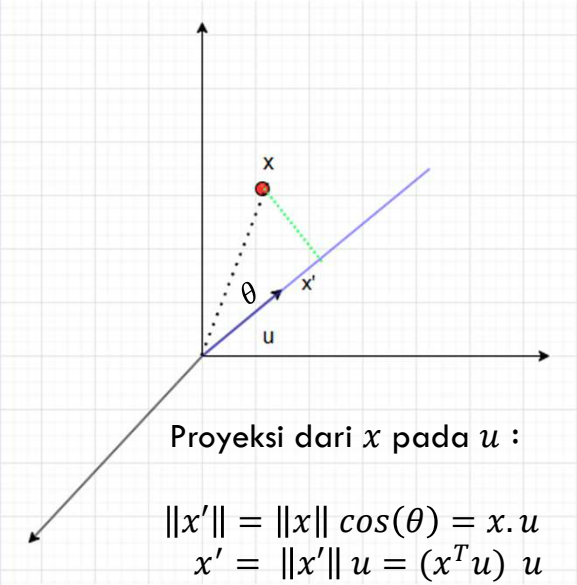
$$x' = (x^T u)\, u$$

$(x^T u)$ adalah besaran dari vector proyek proyeksi dan menunjukkan besaran data x hasil proyeksi pada vector basis u.

- Kita ingin memaksimalkan jumlah $(x^T u)^2$ semua data x pada u di mana $u^T u = 1$

artinya ini persoalan optimasi yaitu kita ingin memaksimalkan fungsi

$$\frac{1}{n}\sum_i (x_i^T u)^2 \text{ dengan batasan } u^T u = 1$$



CARA KERJA PCA

Kita sederhanakan

$$\frac{1}{n}\sum_i (x_i^T u)^2 =$$

$$\frac{1}{\mathrm{n}}\sum_i (x_i^T u)^T\, x_i^T u =$$

$$\frac{1}{n}\sum_i u^T x_i\, x_i^T u =$$

$$\frac{1}{n} u^T \sum_i x_i\, x_i^T u = \boldsymbol{u^T C u}$$

di mana $\boldsymbol{C = \frac{1}{n}\sum_i x_i\, x_i^T}$

Persoalan optimasi menjadi :

$$u^T C u \text{ dan } u^T u = 1$$

Di mana C adalah matrix covariant

Dengan menggunakan teknik *Lagrange Multiplier* kita bisa menyelesaikan persoalan optimasi di atas

Kita optimasi fungsi lagrange :

$$u^T C u - \lambda\,(u^T u - 1)$$

Ambil gradient pada u dan cari nilai optimal

$$2Cu - 2\lambda u = 0$$

Sehingga :

$$\boldsymbol{Cu = \lambda u}$$

CARA KERJA PCA

- Persamaan $\boldsymbol{Cu} = \boldsymbol{\lambda u}$ dikenal problem eigen
- Diketahui matriks C adalah matriks co-variant yang berasal dari data x, yaitu

$$\boldsymbol{C} = \frac{\mathbf{1}}{\boldsymbol{n}} \sum_{\boldsymbol{i}} \boldsymbol{x_i}\, \boldsymbol{x_i^T}$$

Kita ingin mencari sekumpulan vektor eiger u dan nilai eigen λ

- Sehingga kita bisa menggunakan teknik penyelesaian problem eigen

$$Cu - \lambda u = 0$$
$$(C - \lambda I)\, u = 0$$

di sini I adalah matriks identitas dengan bentuk yang sama dengan matriks C. Kondisi di atas hanya akan berlaku jika $(C - \lambda I)$ tidak ada inverse-nya (matriks singular). Artinya :

$$|\boldsymbol{C} - \boldsymbol{\lambda I}| = \mathbf{0}$$

Menyelesaikan persamaan determinan matriks di atas menghasilkan nilai eigen λ dan kemudian setiap nilai eigen menghasilkan vektor eigen u

CARA KERJA PCA

- Vektor eigen u adalah komponen utama (*Principal Component*) yang dicari.
- Nilai eigen mewakili varians yang ditangkap oleh setiap PC.
- Pertahankan hanya 2-3 unit vektor teratas (atau cukup untuk menangkap ~95% dari varians).
- Proyeksikan data ke unit-unit vektor ini untuk mendapatkan versi dimensi rendah yang disederhanakan.

ALGORITMA PCA

Diberikan sebuah dataset berdimensi tinggi dalam bentuk matriks X

1. Lakukan standardisasi data yang efeknya adalah menggeser pusat data ke arah rata-rata data (mean) untuk setiap fitur. Gunakan normalisasi dengan z-score

$$z = \frac{x - \mu}{\sigma}$$

   Di mana μ adalah rata-rata dari setiap fitur, $\mu = \{\mu_1, \mu_2, \dots, \mu_m\}$

   σ adalah deviasi standar setiap fitur, $\sigma = \{\sigma_1, \sigma_2, \dots, \sigma_m\}$

   hasilnya adalah dataset Z dengan mean $\mu' = 0$ dan $\sigma' = 1$

2. Buat matriks Kovarians dari data Z

Matriks kovarian Σ menghitung seberapa besar hubungan perubahan 2 variabel atau lebih.

$$\Sigma = \frac{1}{n-1} Z^T Z$$

ALGORITMA PCA

Z adalah matriks data yang sudah terstandardisasi

Z^T adalah matriks transpose dari Z

Interpretasi dari matrik kovarian Σ :

- **Diagonal utama**: Varian dari fitur ke i (σ_{ii}^2).
- **Elemen non-diagonal**: Kovarian antar-fitur ke i dan j (σ_{ij}^2).
  - ✓ Nilai positif ⟶ Berkorelasi positif.
  - ✓ Nilai negatif ⟶ Berkorelasi negatif.
  - ✓ Nilai 0 ⟶ Tidak berkorelasi.

$$\begin{bmatrix} Cov(x,x) & Cov(x,y) & Cov(x,z) \\ Cov(y,x) & Cov(y,y) & Cov(y,z) \\ Cov(z,x) & Cov(z,y) & Cov(z,z) \end{bmatrix}$$

Contoh matrix Σ 3x3

Contohnya jika elemen $\Sigma[1,0] = 0.8$ artinya fitur ke 1 dan ke 2 berkorelasi kuat.

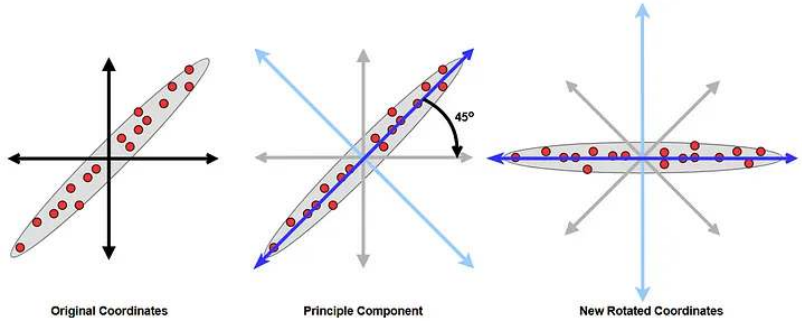
Jika dimensi dari fitur sangat tinggi, komputasi Σ memakan memori dan waktu.

ALGORITMA PCA

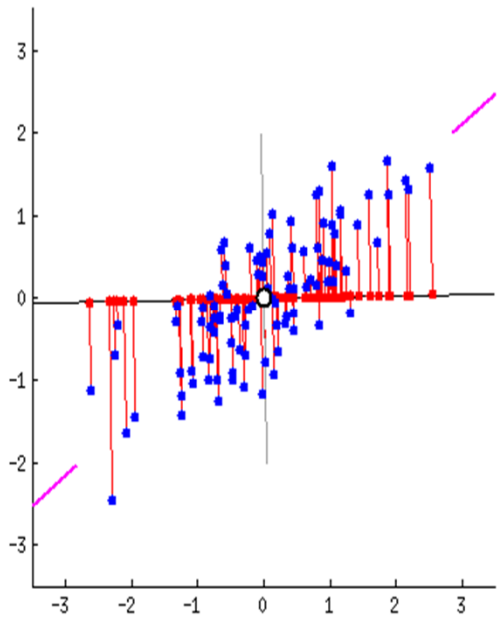
3. Temukan vektor basis sebagai komponen utama dengan menggunakan penyelesaian persamaan problem eigen untuk matriks kovarian

$$|\Sigma - \lambda I| = 0$$

3. Ambil k buah vektor basis yang paling tinggi yang behubungan dengan nilai eigen terbesar. Biasanya k adalah 2 atau 3 karena sudah cukup menangkap 95 % dari variasi data



ALGORITMA PCA



- Komponen utama pertama memperhitungkan varians terbesar yang mungkin dalam kumpulan data.
- Garis yang cocok dengan tanda ungu karena melewati titik asal dan itu adalah garis di mana proyeksi titik-titik (titik merah) paling menyebar.
- Secara matematis, itu adalah garis yang memaksimalkan varians

KEUNTUNGAN PCA

1. **Penanganan Multikolinearitas:** Membuat variabel baru yang tidak berkorelasi untuk mengatasi masalah ketika fitur asli sangat berkorelasi.

2. **Mengurangi Derau:** Menghilangkan komponen dengan varians rendah (diasumsikan sebagai kebisingan) sehingga meningkatkan kejelasan data riil.

3. **Kompresi Data:** Mendeskripsi data dengan lebih sedikit komponen sehingga mengurangi kebutuhan penyimpanan dan mempercepat pemrosesan.

4. **Deteksi Outlier:** Mengidentifikasi titik data yang tidak biasa dengan menunjukkan titik data yang menyimpang secara signifikan dalam ruang yang diperkecil.

KEKURANGAN PCA

1. **Tantangan Interpretasi:** Komponen baru merupakan kombinasi variabel asli, yang mungkin sulit dijelaskan.

2. **Sensitivitas Penskalaan Data:** Memerlukan penskalaan data yang tepat sebelum penerapan, atau hasilnya mungkin menyesatkan.

3. **Kehilangan Informasi:** Mengurangi dimensi dapat menghilangkan beberapa informasi penting jika terlalu sedikit komponen yang disimpan.

4. **Asumsi Linearitas:** Berfungsi paling baik ketika hubungan antara variabel bersifat linear, dan mungkin bermasalah dengan data non-linier.

5. **Kompleksitas Komputasi:** Dapat menjadi lambat dan membutuhkan banyak sumber daya pada kumpulan data yang sangat besar.

6. **Risiko Overfitting:** Menggunakan terlalu banyak komponen atau bekerja dengan kumpulan data yang kecil dapat menyebabkan model tidak dapat digeneralisasi dengan baik.

SETELAH REDUKSI DIMENSI

- Selain dilakukan plot data pada dimensi yg bisa divisualisasi (2D atau 3D), apa yang dilakukan pada data yang sudah direduksi dimensinya?
- Kita bisa melakukan klasifikasi pada data yang sudah ditransformasi tersebut
- Performa model klasifikasi **bisa menjadi lebih baik** setelah reduksi dimensi, terutama jika:

1. Data asli mengandung **banyak noise atau redundansi**.
2. Jumlah fitur sangat besar dibanding sampel (*curse of dimensionality*).
3. Reduksi dimensi dilakukan dengan metode yang tepat (sesuai karakteristik data).

Namun, hasilnya tergantung pada **metode reduksi, algoritma klasifikasi,** dan **sifat dataset**

CONTOH IMPLEMENTASI DENGAN SKLEARN

- Dalam modul sklearn.decomposition terdapat kelas **PCA** yang mengimplementasikan algoritma PCA
- Berikut adalah contoh implementasi dengan dataset bunga Iris.
- Ada 3 tipe bunga Iris : *Sentosa, Virginica* dan *Versicolor*
- Data memiliki 4 buah fitur :
  - ✓Sepal.Length
  - ✓Sepal.Width
  - ✓Petal.Length
  - ✓Petal.Width
- Jumlah total data : 150

Sentosa

Versicolor

Virginia

CONTOH IMPLEMENTASI DENGAN SKLEARN

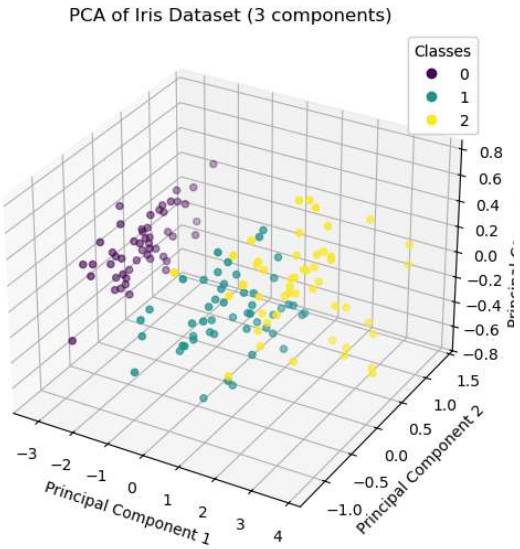
- Import paket, class dan fungsi
- Load dataset iris

```
iris = load_iris()
X = iris.data
y = iris.target
```

- Definisikan model dan lakukan transformasi PCA dengan 3 buah PC

```
pca = PCA(n_components=3)
X_pca = pca.fit_transform(X)
```

- Data sudah dilakukan reduksi dimensi dari 4 fitur menjadi 3 buah fitur yang merupakan principal component dari data.
- Plot Data X_pca, karena ada 3 PC maka dilakukan proyeksi 3 D



KESIMPULAN

- Data berdimensi tinggi (high-dimensional data) mengacu pada dataset yang memiliki sejumlah besar fitur atau variabel. Ini menimbulkan tantangan pada machine learning karena peningkatan kompleksitas dan kebutuhan komputasi yang lebih besar.

- Reduksi Dimensi adalah salah satu cara mengatasi problem data berdimensi tinggi

- PCA adalah salah satu metode mereduksi dimensi dengan asumsi data bersifat linier atau komponen hasil reduksi adalah kombinasi linier dari fitur-fitur yang ada

- Prinsip PCA adalah menemukan vektor basis yang dapat merepresentasikan data dengan baik. Vektor basis inilah disebut dengan principal component (PC).

- Pembentukan Matriks Kovarian dari data adalah bagian penting dari algoritma PCA di mana Principal Component didapat dari penyelesaian problem eigen.